



RO:BIT
ROBOT SPORT GAME TEAM

Team. RO : BIT | **신입생 교육
Python 1일차**

RO : BIT 16th
지능형로봇팀
백인엽

INDEX

1. Python 소개
2. 환경 설정
3. 기초 문법
4. 과제

| Python 소개



-특징

1. 플랫폼 독립적
2. 인터프리터식 언어
3. 객체 지향적 언어
4. 동적 타이핑

-장점

1. 문법이 쉽고 인간친화적 언어
2. 라이브러리 접근 및 사용이 편리
3. 개발속도가 빠르다
4. 인공지능 및 데이터 분석 생태계 강력

-단점

1. 코드 실행속도 느림
2. 메모리 사용량이 크다
3. 라이브러리 및 환경 관리 문제

Python 소개

Jul 2026	Jul 2025	Change	Programming Language		Ratings	Change
1	1			Python	18.94%	-8.03%
2	3	⬆		C	10.86%	+1.22%
3	2	⬇		C++	9.12%	-0.68%
4	4			Java	8.03%	-0.73%
5	5			C#	4.49%	-0.38%
6	6			JavaScript	2.72%	-0.63%
7	8	⬆		Visual Basic	2.48%	+0.54%
8	13	⬆⬆		SQL	1.71%	+0.32%
9	15	⬆⬆		R	1.69%	+0.44%
10	18	⬆⬆		Rust	1.34%	+0.33%

-TIOBE

Python 소개

-Python은 단순히 쉬운 언어?

-파이썬은 혼자 다 하는 언어라기보다, 다른 기술을 연결하는 언어에 가깝다

파이썬이 인공지능이나 데이터 분석에서 많이 쓰이는 이유는 파이썬 자체가 모든 계산을 가장 빠르게 처리해서가 아니다. 실제로 무거운 계산은 C, C++, CUDA 같은 빠른 언어와 기술로 구현되어 있고, 파이썬은 그것을 사용하기 쉽게 연결해주는 역할을 한다.

-파이썬은 문법보다 생태계가 강한 언어이다

파이썬의 진짜 강점은 문법이 쉬운 것만이 아니라, 이미 만들어져 있는 라이브러리와 커뮤니티가 매우 크다는 점이다.

우리가 어떤 기능을 만들고 싶을 때, 대부분의 경우 처음부터 전부 직접 구현할 필요가 없다. 데이터 분석은 pandas, 수치 계산은 NumPy, 인공지능은 PyTorch, 영상처리는 OpenCV, 웹 서버는 Flask나 FastAPI처럼 분야별로 널리 사용되는 도구들이 존재한다.

-파이썬 교육의 나름의 목표

AI와 LLM의 발전으로 이제 특정 언어의 문법을 달달 외우고 마스터 한다는 것은 크게 의미가 없다. 기초적인 내용을 배우고 알고리즘의 구현 흐름이나 최적의 오픈소스 활용법 같은 논리적 과정을 생각해보는 시간이 되었으면 함.

환경 설정

-VSCode Extension



Python pre-release

Microsoft  microsoft.com |  226,650,573 |      (630)

Python language support with extension access points for IntelliSense (Pylance), Debugging (Python Debugger), linting, formatting, refactoring, unit tests, and more.

Restart Extensions

Switch to Release Version

Disable ▾

Uninstall ▾

☒ Auto Update 



Pylance

Microsoft  microsoft.com |  193,928,557 |     (278)

A performant, feature-rich language server for Python in VS Code

Switch to Pre-Release Version

Disable ▾

Uninstall ▾

☒ Auto Update 

지금 진행하는 환경 설정은 파이썬 코딩 자체가 처음일 경우 따라 진행해서 환경 구축을 하고
자신이 기존에 쓰던 세팅이나 환경이 있으면 그걸 그대로 사용해도 무방함

환경 설정

-프로젝트 폴더 생성

Windows PowerShell 사용

```
PS C:\Users\HOME> mkdir C:\robit_python
```

디렉터리: C:\

Mode	LastWriteTime	Length	Name
d-----	2026-07-09 오후 7:52		robit_python

```
PS C:\Users\HOME> cd C:\robit_python\  
PS C:\robit_python>
```

mkdir C:\robit_python

cd C:\robit_python

```
PS C:\robit_python> code .
```

code .

직접 폴더 생성

> 내 PC > 로컬 디스크 (C:) > robit_python

-> 해당 경로에서 우클릭 후 '터미널에서 열기'

환경 설정

-가상환경 생성

Power Shell 명령어

```
py -m venv .venv
```

```
.\venv\Scripts\Activate.ps1
```

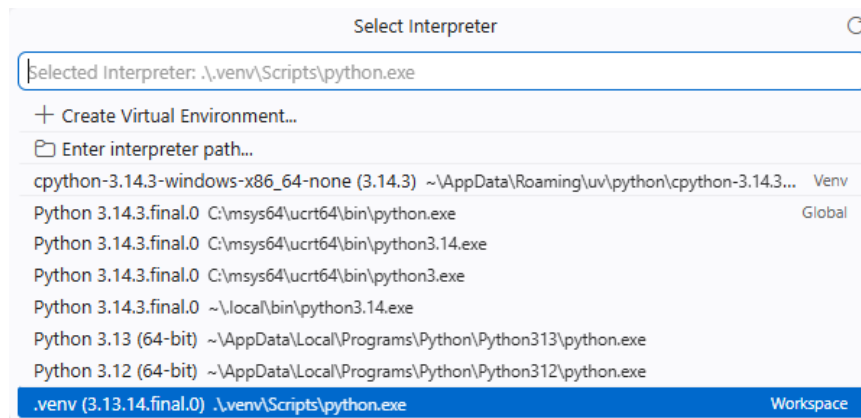
```
PS C:\robit_python> py -m venv .venv
PS C:\robit_python> .\venv\Scripts\Activate.ps1
(.venv) PS C:\robit_python> |
```

만약 안되면

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

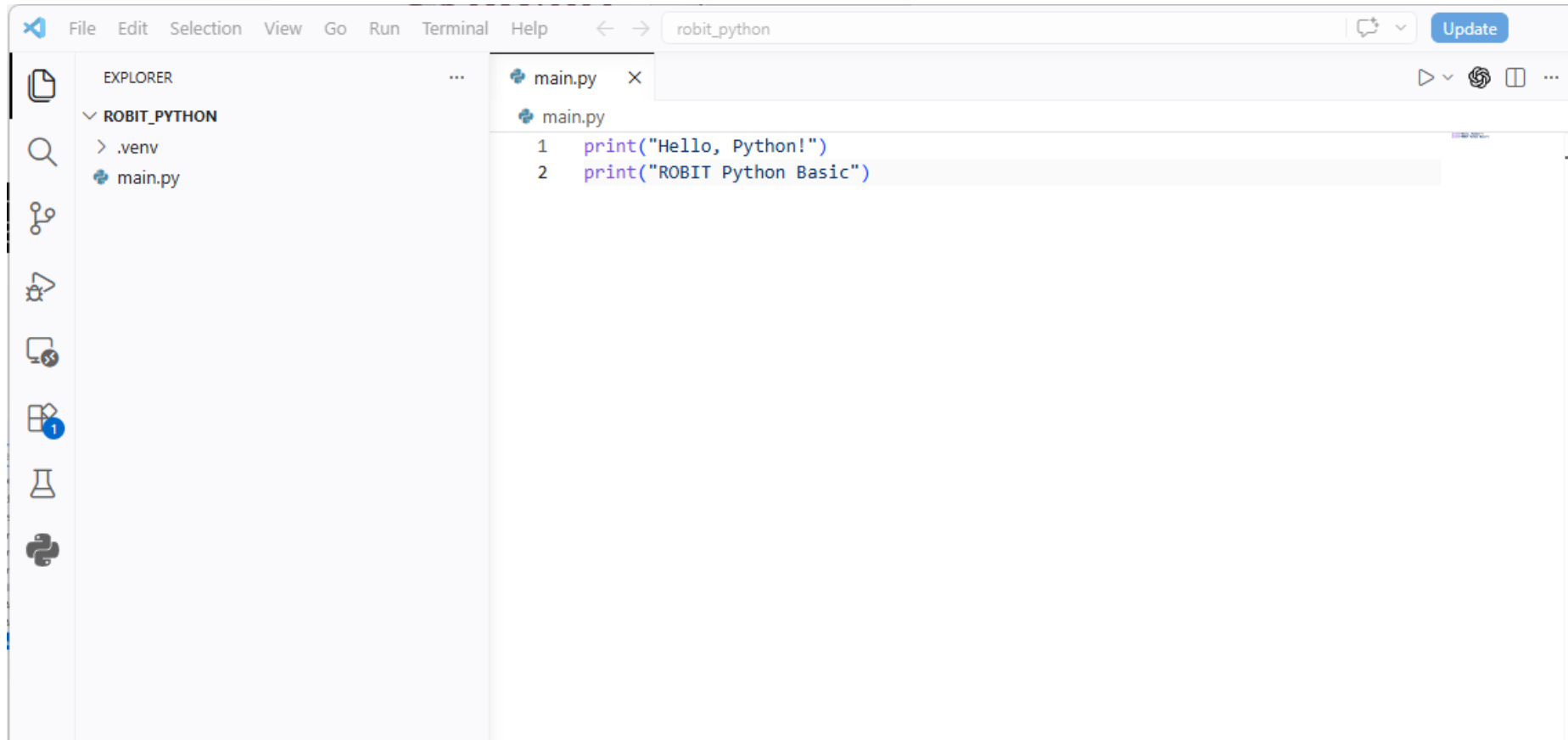
VSCode 설정

1. Ctrl + Shift + P
2. Python: Select Interpreter
3. .venv(3.13.4.final.0) .\venv\Scripts\python.exe



환경 설정

-파이썬 파일 생성



내용 저장 후 터미널에서 `python main.py`를 실행하여 제대로 실행되는지 확인

기초 문법

-print와 f-string

- print()는 값을 화면에 보여주는 함수이다.
- 쉼표로 여러 값을 넣으면 자동으로 공백이 들어간다.
- f-string은 문자열 안에 변수 값을 끼워 넣는 방식이다.
- 출력이나 디버깅할 때 변수 상태를 확인하는 가장 기본적인 도구이다.

```
name = "홍길동"
age = 24

print("이름:", name)
print(f"이름: {name}, 나이: {age}")
```

```
• (.venv) PS C:\robit_python> python main.py
이름: 홍길동
이름: 홍길동, 나이: 24
```

유의할 점

print는 값을 “반환”하는 것이 아니라 화면에 “출력”한다. 나중에 함수의 return을 배울 때 이 차이가 중요하다.

실전 활용

센서값, 변수값, 조건문 진입 여부를 확인할 때 print를 자주 사용한다. 복잡한 코드가 안 될 때는 먼저 값이 예상대로 흐르는지 출력해 본다.

기초 문법

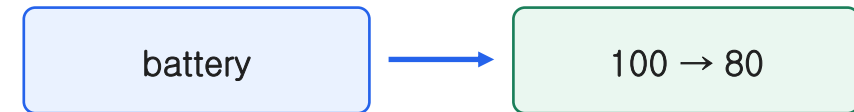
-변수

- 변수는 값을 저장하는 상자가 아니라, 값에 붙인 이름표로 이해하면 좋다.
- `=` 는 같다라는 뜻이 아니라 오른쪽 값을 왼쪽 이름에 연결한다는 뜻이다.
- 변수의 값은 이후 줄에서 바뀔 수 있다.
- 변수 이름은 코드의 의미를 전달하는 첫 번째 설명이다.

```
battery = 100  
print(battery)
```

```
battery = 80  
print(battery)
```

```
● (.venv) PS C:\robit_python> python main.py  
100  
80
```



유의할 점

`battery = battery - 10` 같은 코드는 “같다”가 아니라 현재 `battery` 값을 읽고, 10을 뺀 새 값을 다시 `battery`라는 이름에 연결한다는 뜻이다.

기초 문법

-변수

- 숫자로 시작할 수 없다.
- 하이픈(-)은 뿔셈으로 해석되므로 변수명에 쓰지 않는다.
- 예약어는 변수명으로 사용할 수 없다.
- 파이썬에서는 snake_case를 많이 사용한다.

```
robot_name = "robit_bot"
battery_level = 90
psd_distance = 2.5

# 비추천
a = 90
x1 = 2.5

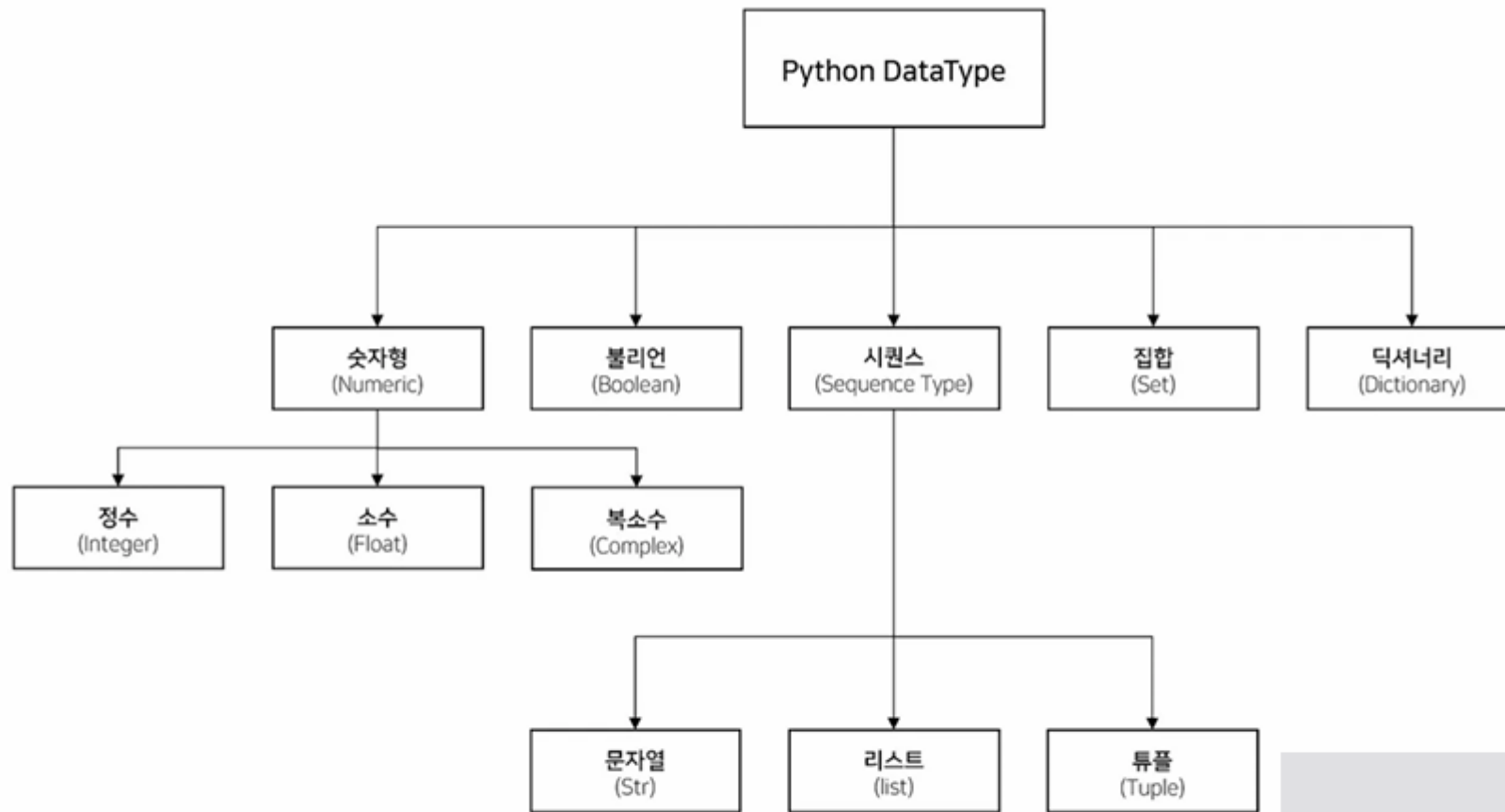
# 불가
if = 1 #예약어
for = 10
num-1 = 10 #num_1 = 10
```

실전 활용

프로그램이 길어질수록 “동작하는 코드”보다 “다시 읽을 수 있는 코드”가 중요해진다. 변수명은 미래의 나와 팀원을 위한 설명이다.

기초 문법

-자료형과 형변환



파이썬의 특징 : 동적 타이핑 - 동적 타이핑이란 변수의 자료형을 미리 선언하지 않고, 실행 중에 값에 따라 자료형이 결정되는 방식

기초 문법

-자료형과 형변환

- input()으로 들어온 값은 항상 문자열이다.
- 계산하려면 int() 또는 float()으로 바꿔야 한다.
- 문자열로 합치려면 str()로 바꿀 수 있다.
- 형변환은 실패할 수도 있다. 예: int("abc")

```
age = input("나이: ")  
print(type(age)) # str
```

```
age = int(age)  
print(age + 1)
```

int
정수

float
실수

str
문자열

bool
참/거짓

유의할 점

사용자가 숫자가 아닌 값을 입력하면 형변환에서 에러가 난다. 실제 프로그램에서는 예외 처리로 입력 검사를 추가한다.

```
• (.venv) PS C:\robit_python> python main.py  
나이: 24  
<class 'str'>  
25
```

기초 문법

-연산자

- 연산자는 값 사이에서 계산, 비교, 판단을 수행하는 기호 또는 키워드이다.

- 산술 연산자
- 대입 연산자
- 비교 연산자
- 논리 연산자
- 문자열 연산자
- 멤버십 연산자
- 비트 연산자

기초 문법

-연산자 (산술 연산자)

연산자	의미	예시	결과
+	덧셈	10 + 3	13
-	뺄셈	10 - 3	7
*	곱셈	10 * 3	30
/	나눗셈	10 / 3	3.333...
//	몫	10 // 3	3
%	나머지	10 % 3	1
**	거듭제곱	10 ** 3	1000

```
a = 10
b = 3

print(a / b) # 3.333...
print(a // b) # 3
print(a % b) # 1
print(a ** b) # 1000
```

```
● (.venv) PS C:\robit_python> python main.py
3.3333333333333335
3
1
1000
```


기초 문법

-연산자 (대입 연산자, 복합 대입 연산자)

연산자	의미	예시
+=	덧셈 후 저장	num += 3
-=	뺄셈 후 저장	num -= 3
*=	곱셈 후 저장	num *= 3
/=	나눗셈 후 저장	num /= 3
//=	몫 저장	num //= 3
%=	나머지 저장	num %= 3
**=	거듭제곱 저장	num **= 3

```
count = 0
x = 10

count += x
print(count)
count -= x
print(count)
```

```
● (.venv) PS C:\robit_python> python main.py
10
0
```

기초 문법

-연산자 (비교 연산자)

연산자	의미	예시
==	같다	x == 10
!=	같지 않다	x != 10
>	크다	x > 10
<	작다	x < 10
>=	크거나 같다	x >= 10
<=	작거나 같다	x <= 10

유의할 점

비교 연산자의 결과는 항상 불리언 값이다. (True or False)
=와 ==를 잘 구별하자

```
score = 85

if score >= 80:
    print("합격")
else:
    print("불합격")
```

```
● (.venv) PS C:\robit_python> python main.py
합격
```

기초 문법

-연산자 (논리 연산자)

and

둘 다 참이어야 참

or

하나라도 참이면 참

not

참/거짓을 반대로

```
x = 10
y = 20

print(x > 0 and y > 10)

print(x > 0 or y < 0)

print(not x > 0)
```

```
● (.venv) PS C:\robit_python> python main.py
True
True
False
```

기초 문법

-연산자 (문자열 연산자)

연산자	의미	예시	결과
+	문자열 연결	"Py" + "thon"	"Python"
*	문자열 반복	"안녕" * 3	"안녕안녕안녕"
[]	인덱싱	"Python"[0]	"P"
[:]	슬라이싱	"Python"[0:2]	"Py"

```
first = "Hello"
second = "Python"

print(first + " " + second)
print("Python " * 3)
```

유의할 점

문자열과 숫자는 바로 더할 수 없다. 만약 문자열 반복이 아닌 숫자를 이어 붙이고 싶다면 형변환을 사용하면 된다.

```
• (.venv) PS C:\robit_python> python main.py
Hello Python
Python Python Python
```

| 기초 문법

-연산자 (멤버십 연산자)

in

포함되어 있다

not in

포함되어 있지 않다

```
fruits = ["apple", "banana", "grape"]
```

```
print("apple" in fruits)
```

```
print("orange" not in fruits)
```

```
text = "Hello Python"
```

```
print("Python" in text)
```

```
● (.venv) PS C:\robit_python> python main.py  
True  
True  
True
```

기초 문법

-연산자 (비트 연산자)

연산자	의미
&	AND
	OR
^	XOR
~	NOT
<<	왼쪽 시프트
>>	오른쪽 시프트

& 연산 결과 예시

```
0101
0011
----
0001
```

```
a = 5  # 0101
b = 3  # 0011
```

```
print(a & b)
print(a | b)
print(a ^ b)
```

실전 활용

보통 센서 상태값, 모터 드라이버 플래그, 통신 패킷의 특정 비트 확인 등에 사용될 수 있다.

```
● (.venv) PS C:\robit_python> python main.py
1
7
6
```

기초 문법

-조건문

- 조건문은 '조건이 참인지 거짓인지'에 따라 실행 흐름을 나누는 문법이다.
- 즉, 상황에 따라 다른 코드를 실행할 수 있게 해준다.
- 파이썬에서는 주로 if, elif, else를 사용한다.
- 조건문은 비교 연산자(==, !=, >, <, >=, <=) 및 논리 연산자(and, or, not)와 함께 자주 사용된다.

if 문

```
score = 85  
  
if score >= 80:  
    print("합격")
```

if else 문

```
score = 85  
  
if score >= 80:  
    print("합격")  
else:  
    print("불합격")
```

elif

```
score = 85  
  
if score >= 80:  
    print("A")  
elif score >= 60:  
    print("B")  
else:  
    print("C")
```

기초 문법

-중첩 조건문

- 조건문 내부에 또다른 조건문을 삽입하여 케이스를 나눌 수 있다.

```
age = 20
has_ticket = True

if age >= 18:
    if has_ticket:
        print("입장 가능")
    else:
        print("티켓이 필요합니다")
else:
    print("미성년자는 입장 불가")
```

```
if age >= 18 and has_ticket:
    print("입장 가능")
else:
    print("입장 불가")
```

유의할 점

파이썬에서 다중 중첩된 문법의 경우 들여쓰기로 코드의 범위를 결정한다. 따라서 들여쓰기 된 코드 위치에 따라 다른 문법에 포함되지 않도록 주의.

실전 활용

조건이 복잡해질 수록 논리 연산자 등을 활용해 조건의 단순화가 가능한지 함께 고려해보는 것도 좋다.

기초 문법

-3항 연산자

- 간단한 조건문을 간결하게 작성하는 방식

형태 - 참일 때 값 if 조건식 else 거짓일 때 값

```
score = 85  
  
if score >= 80:  
    print("합격")  
else:  
    print("불합격")
```



```
score = 85  
  
print("합격" if score >= 80 else "불합격")
```

```
a = 10  
b = 20  
  
if a > b:  
    max_val = a  
else:  
    max_val = b  
  
print(max_val)
```



```
a = 10  
b = 20  
  
max_val = a if a > b else b  
print(max_val)
```

기초 문법

-반복문

- 반복문은 같은 동작을 여러 번 실행할 때 사용하는 문법이다.
- 코드를 반복해서 직접 작성하지 않고, 필요한 횟수만큼 자동으로 실행할 수 있다.
- 파이썬에서는 주로 for문과 while문을 사용한다.
- 반복문은 출력, 계산, 리스트 처리, 입력 검증 등 다양한 상황에서 사용된다.

```
print("안녕하세요")  
print("안녕하세요")  
print("안녕하세요")  
print("안녕하세요")  
print("안녕하세요")
```



```
for i in range(5):  
    print("안녕하세요")
```

```
count = 0  
  
while count < 5:  
    print("안녕하세요")  
    count += 1
```

For문

- 반복 횟수가 정해져 있을 때
- 리스트나 문자열의 요소를 하나씩 꺼낼 때

While문

- 반복 횟수보다 종료 조건이 중요할 때
- 사용자가 올바른 값을 입력할 때까지 반복할 때
- 특정 상태가 될 때까지 계속 확인할 때

기초 문법

-반복문

- 파이썬에서의 for문

유의할 점

파이썬에서의 for문을 단순히 숫자를 반복하는 문법으로 이해하면 안됨.
반복 가능한 객체를 하나씩 꺼내는 문법으로 이해하는게 좀 더 정확함.

문자열

```
for char in "ROBIT":  
    print(char)
```

```
• (.venv) PS C:\robit_python> python main.py  
R  
O  
B  
I  
T
```

리스트

```
robots = ["robot1", "robot2", "robot3"]  
  
for robot in robots:  
    print(robot)
```

```
• (.venv) PS C:\robit_python> python main.py  
robot1  
robot2  
robot3
```

기초 문법

-반복문 제어

- break : 반복문을 즉시 종료하고 빠져나옴
- continue : 이번 루프의 나머지 코드를 건너뛰고, 다음 반복으로 넘어감

```
for i in range(1, 6):  
    if i == 3:  
        continue  
    if i == 5:  
        break  
    print(i)
```

```
count = 0  
  
while count < 5:  
    count += 1  
    if count == 3:  
        continue  
    if count == 5:  
        break  
    print(count)
```

유의할 점

range()의 범위 혼동하지 않기
While문의 종료조건 확인(무한반복 가능)

실전 활용

횟수가 명확하면 for, 특정 조건이 만족될 때까지면 while로 생각하면 쉽다.

```
(.venv) PS C:\robit_python> python main.py  
1  
2  
4
```

기초 문법

-중첩 반복문

- 반복문 내부에 또 다른 반복문을 삽입하여 중첩된 반복을 실행할 수 있다.

```
for i in range(3):  
    print("바깥 반복:", i)  
  
    for j in range(2):  
        print("  안쪽 반복:", j)
```

```
● (.venv) PS C:\robit_python> python main.py  
바깥 반복: 0  
  안쪽 반복: 0  
  안쪽 반복: 1  
바깥 반복: 1  
  안쪽 반복: 0  
  안쪽 반복: 1  
바깥 반복: 2  
  안쪽 반복: 0  
  안쪽 반복: 1
```

유의할 점

반복문이 중첩될 수록 계산량이 반복횟수의 곱셈의 형태로 기하급수적으로 늘어날 수 있다.

실전 활용

보통 2차원 구조를 다룰 때 주로 사용한다.
e.g.) 2차원 리스트, 이미지 픽셀처리, 행렬연산, 맵 탐색 등

기초 문법

-리스트

- 여러 개의 값을 하나의 변수에 저장할 수 있는 자료형
- 파이썬 리스트는 하나의 리스트 안에 서로 다른 자료형을 넣을 수 있다.

```
numbers = [1, 2, 3, 4, 5]
names = ["철수", "영희", "민수"]
mixed = [1, "python", 3.14, True]
```

인덱스	값
0	철수
1	영희
2	민수

인덱스 접근

```
fruits = ["apple", "banana", "grape"]

print(fruits[0]) # apple
print(fruits[1]) # banana
print(fruits[2]) # grape
```

음수 인덱스

```
fruits = ["apple", "banana", "grape"]

print(fruits[-1]) # grape
print(fruits[-2]) # banana
print(fruits[-3]) # apple
```

| 기초 문법

-리스트 기능

리스트 값 변경

```
fruits = ["apple", "banana", "grape"]  
fruits[1] = "orange"  
print(fruits)
```

리스트 값 추가(리스트 맨 끝)

```
fruits = ["apple", "banana", "grape"]  
fruits.append("orange")  
print(fruits)
```

리스트 값 추가(특정 위치)

```
fruits = ["apple", "banana", "grape"]  
fruits.insert(2, "orange")  
print(fruits)
```

리스트 길이 반환

```
fruits = ["apple", "banana", "grape"]  
print(len(fruits))
```

리스트 값 삭제(값 기준)

```
fruits = ["apple", "banana", "grape"]  
fruits.remove("banana")  
print(fruits)
```

리스트 값 삭제(인덱스 기준)

```
fruits = ["apple", "banana", "grape"]  
fruits.pop(1)  
print(fruits)
```

기초 문법

-2차원 리스트

2차원 리스트 값 접근

```
matrix = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
  
print(matrix[0][0])  
print(matrix[1][2])  
print(matrix[2][1])
```

```
• (.venv) PS C:\robit_python> python main.py  
1  
6  
8
```

중첩 반복문 예제

```
matrix = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
  
for row in matrix:  
    for value in row:  
        print(value, end=" ")  
    print()
```

```
• (.venv) PS C:\robit_python> python main.py  
1 2 3  
4 5 6  
7 8 9
```

유의할 점

리스트에 없는 인덱스에 접근하면 오류가 발생한다.

기초 문법

-리스트 참조

리스트 참조

```
a = [1, 2, 3]
b = a

b[0] = 100

print(a)
print(b)
```

리스트 복사

```
a = [1, 2, 3]
b = a.copy()

b[0] = 100

print(a)
print(b)
```

유의할 점

리스트를 다른 변수에 대입하면 리스트가 복사되는 것이 아니라, 같은 리스트를 함께 가리키게 된다.

```
• (.venv) PS C:\robit_python> python main.py
[100, 2, 3]
[100, 2, 3]
```

```
• (.venv) PS C:\robit_python> python main.py
[1, 2, 3]
[100, 2, 3]
```

기초 문법

-튜플(tuple)

```
fruits = ("apple", "banana", "grape")

print(fruits[0])
print(fruits[1])
print(fruits[2])

fruits[1] = "orange"

print(fruits)
```

유의할 점 1

튜플은 리스트와 비슷해 보이지만 값이 변경 가능한 리스트와 달리 튜플의 값은 변경할 수 없다.

유의할 점 2

기본적으로 튜플의 값은 변경이 불가하지만 튜플 내부에 리스트 같은 mutable 객체가 들어가면 내부 리스트는 바뀔 수 있다.

실전 활용

좌표값, RGB 고정 색상값, 기타 고정 설정값 등 변하면 안되는 데이터를 표현할 수 있다.

```
• (.venv) PS C:\robit_python> python main.py
apple
banana
grape
Traceback (most recent call last):
  File "C:\robit_python\main.py", line 7, in <module>
    fruits[1] = "orange"
    ~~~~~^~
TypeError: 'tuple' object does not support item assignment
```

기초 문법

-딕셔너리

- 딕셔너리는 key와 value를 한 쌍으로 저장한다.
- 순서보다 의미 있는 이름으로 접근하는 것이 중요하다.
- 하나의 대상이 여러 속성을 가질 때 유용하다.
- 로봇 정보, 설정값, 센서 상태처럼 구조화된 데이터에 자주 사용된다.

```
robot = {  
    "name": "frontbot",  
    "battery": 80,  
    "sensor": "lidar"  
}  
  
print(robot["battery"])
```



유의할 점

리스트는 robots[0]처럼 위치로 접근하지만, 딕셔너리는 robot["battery"]처럼 의미로 접근한다. 딕셔너리도 순서를 유지하긴 하지만, 핵심은 순서가 아니라 key를 통해 의미 있는 값에 접근한다

실전 활용

딕셔너리는 자유도가 높지만 그만큼 오타에 취약하다. 프로젝트에서는 key 이름을 문서화하거나, 나중에는 클래스를 사용해 구조를 더 명확하게 만들 수 있다.

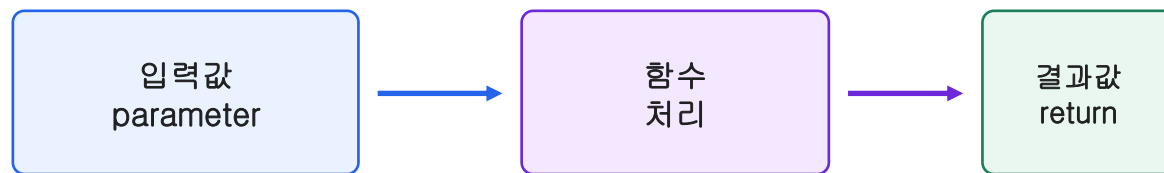
기초 문법

-함수(function)

- 함수는 특정 기능을 수행하는 코드 묶음이다.
- 같은 코드를 여러 번 복사하지 않아도 된다.
- 기능 단위로 나누면 코드가 읽기 쉬워진다.
- 입력값을 받고 결과값을 돌려주는 구조로 생각하면 쉽다.

```
def move_forward(name):  
    print(f"{name} 로봇 전진")  
  
def stop_robot(name):  
    print(f"{name} 로봇 정지")  
  
move_forward("로봇1")  
stop_robot("로봇1")  
move_forward("로봇2")  
stop_robot("로봇2")
```

```
(.venv) PS C:\robit_python> python main.py  
로봇1 로봇 전진  
로봇1 로봇 정지  
로봇2 로봇 전진  
로봇2 로봇 정지
```



기초 문법

-함수(function)

- return은 함수의 결과값을 함수 바깥으로 돌려주는 역할을 한다.
- return을 만나면 함수는 즉시 종료된다.
- 함수 내부에 return이 없는 경우 자동으로 None을 반환한다.

```
def add(a, b):  
    return a + b
```

```
result = add(3, 5)  
print(result)
```

유의할 점 1

return과 print는 완전히 다르다. print는 단순히 콘솔상에서 보여주는 역할이고 return은 함수 외부에서 값을 사용하기 위한 것이다.

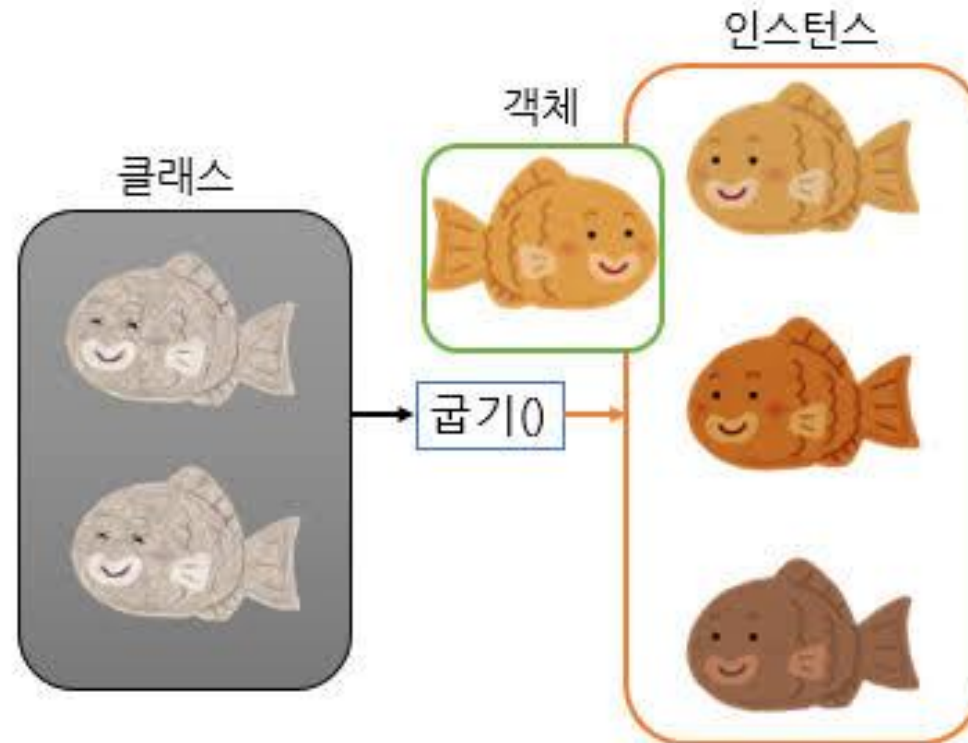
유의할 점 2

함수를 사용할 때는 변수의 범위를 잘 고려해야한다. 변수가 선언된 위치에 따라 전역 변수, 지역변수로 나뉘고 이에 따라 변수에 접근할 수 있는 범위가 달라진다.

기초 문법

-클래스

- 클래스는 객체를 만들기 위한 설계도이다.
- 클래스를 통해 만들어진 실제 대상을 객체 또는 인스턴스라고 한다.
- 객체는 속성(attribute)과 동작(method)을 함께 가질 수 있다.
- 객체란 데이터와 기능을 함께 가진 하나의 대상이다. (e.g. 로봇 - 배터리, 센서값, 속도, 위치 등)
- 인스턴스는 클래스를 기반으로 만들어진 객체를 의미



기초 문법

-클래스

class의 기본 구조 예시

```
class Robot:
    def __init__(self, name, battery):
        self.name = name
        self.battery = battery

    def move_forward(self):
        print(f"{self.name} 전진")

    def show_battery(self):
        print(f"{self.name} 배터리: {self.battery}%")

robot = Robot("kubo", 80)

robot.move_forward()
robot.show_battery()
```

- 속성(attribute) : 객체가 가지고 있는 데이터
- 메서드(method) : 객체가 수행할 수 있는 기능, 함수
- __init__() 메서드
 - 객체가 생성될 때 자동으로 실행되는 메서드. 따라서 보통 객체가 처음 만들어질 때 필요한 초기값을 설정하는데 사용.
- self
 - self는 현재 객체 자기자신을 의미한다.
 - 클래스를 통해 여러 객체가 생성되더라도 self를 통해 각각의 개체가 자기만의 데이터를 가지고 접근할 수 있다.

```
• (.venv) PS C:\robit_python> python main.py
kubo 전진
kubo 배터리: 80%
```

기초 문법

-클래스

인스턴스, 클래스 변수

```
class Robot:
    robot_type = "mobile robot"

    def __init__(self, name):
        self.name = name
```

- 클래스 변수
 - 클래스의 모든 객체가 함께 공유하는 변수
- 인스턴스 변수
 - 객체마다 따로 가지는 변수

기초 문법

-클래스 상속

클래스 상속 예시

```
class Robot:
    def __init__(self, name, battery):
        self.name = name
        self.battery = battery

    def show_battery(self):
        print(f"{self.name} 배터리: {self.battery}%")

class MobileRobot(Robot):
    def move_forward(self):
        print(f"{self.name} 전진")

class ManipulatorRobot(Robot):
    def grasp(self):
        print(f"{self.name} 물체 잡기")

mobile = MobileRobot("wheelbot", 80)
arm = ManipulatorRobot("armbot", 90)

mobile.show_battery()
mobile.move_forward()

arm.show_battery()
arm.grasp()
```

• 상속

- 기존 클래스의 속성과 메소드를 바탕으로 새로운 클래스를 만드는 것
- 공통적인 기능은 상위 클래스(부모 클래스)에 두고 세부 기능은 하위 클래스(자식 클래스)에 두어 추가, 수정, 유지 보수를 원활하게 하는 방식

실전 활용

상속을 활용하면 중복되는 코드를 효과적으로 제거하고 변수 관리 및 기능의 수정, 추가를 효율적으로 할 수 있다.

기초 문법

-클래스 상속 - 메서드 오버라이딩

오버라이딩 예시

```
class Robot:
    def move(self):
        print("로봇이 이동합니다.")

class MobileRobot(Robot):
    def move(self):
        print("바퀴로 이동합니다.")

class Drone(Robot):
    def move(self):
        print("프로펠러로 비행합니다.")
```

• 메서드 오버라이딩

- 자식 클래스에서 부모 클래스와 같은 이름의 메서드를 다시 만들면 자식 클래스의 메서드가 우선 실행된다.

실전 활용

만약 자식클래스에서 오버라이딩한 부모클래스의 메서드가 같이 필요하다면 `super()`를 이용하면 된다.

e.g., `super().move()`

기초 문법

-클래스 상속 - 활용 방식 선정(is-a, has-a)

- 상속을 사용해야할지 판단하는데 있어 가장 간편한 개념

상속 사용 추천

MobileRobot is a Robot

Drone is a Robot

ManipulatorRobot is a Robot

상속 사용 비추천

Robot has a Motor

Robot has a Camera

Robot has a Battery

과제-1

-로봇 상태 데이터 분석

```
robot_status = [  
    {"name": "mobilebot", "battery": 82, "position": (1.2, 0.5), "distance": 0.8},  
    {"name": "drone", "battery": 18, "position": (0.3, 1.5), "distance": 0.4},  
    {"name": "manipulator", "battery": 45, "position": (2.0, 1.0), "distance": 1.2},  
]
```

위의 robot_status를 그대로 사용하여

로봇 이름

배터리 상태

장애물 감지 여부

현재 위치

4가지 정보를 출력하는 프로그램을 작성하시오

출력 예시

[mobilebot]

배터리: 배터리 충분

위치: x=1.2, y=0.5

상태: 전진 가능

평가 기준

- 리스트 안의 딕셔너리 구조를 이해했는가
- 튜플 언패킹을 사용할 수 있는가
- 조건문으로 상태를 분류할 수 있는가
- 반복문으로 여러 데이터를 처리할 수 있는가
- 함수를 이용해 코드를 기능별로 나눌 수 있는가

배터리 상태 기준

battery >= 60 → 배터리 충분

20 <= battery < 60 → 배터리 주의

battery < 20 → 충전 필요

장애물 감지 기준

distance < 0.5 → 장애물 감지

distance >= 0.5 → 전진 가능

과제-2

-코드 수정하기

```
def add_log(robot_name, battery, logs=[]):  
    log = robot_name + " battery: " + battery  
    logs.append(log)  
    return logs  
  
print(add_log("frontbot", 80))  
print(add_log("rearbot", 50))  
print(add_log("armbot", 20))
```

평가 기준

- TypeError를 이해하고 수정할 수 있는가
- f-string 또는 str() 형변환을 사용할 수 있는가
- 함수의 기본값 매개변수 함정을 이해했는가
- 리스트가 mutable 자료형이라는 것을 이해했는가
- 단순히 코드를 고치는 것이 아니라 이유를 설명할 수 있는가

위 코드는 로봇 배터리 로그를 저장하는 프로그램이다.

위 코드의 실행 오류와 논리 오류를 설명하고 수정하시오(코드 내에 주석으로 설명)

1. 실행 시 발생하는 오류를 수정할 것
2. 배터리 값이 문자열과 정상적으로 합쳐지도록 수정할 것
3. 함수 호출마다 로그가 의도치 않게 누적되는 문제를 해결할 것
4. 수정한 이유를 설명할 것

과제-3

-리스트 메서드 구현하기

사용자로부터 명령어를 입력받아 리스트를 조작하는 프로그램을 작성하시오

append 값	리스트 맨 뒤에 값 추가
insert 인덱스 값	특정 위치에 값 추가
remove 값	특정 값 삭제
pop 인덱스	특정 위치 값 삭제
len	리스트 길이 출력
print	리스트 전체 출력
clear	리스트 초기화

입,출력 예시

```
append apple
append banana
insert 1 orange
remove apple
print
len
```

```
['orange', 'banana']
2
```

위 명령어를 지원해야함

고급조건

- 리스트가 비어있을 때 pop을 실행하면 오류가 나지 않도록 예외처리할 것
- 존재하지 않는 값을 remove하려하면 안내 메시지를 출력
- 인덱스 범위를 벗어나면 안내 메시지를 출력

평가 기준

- 리스트의 메서드를 이해했는가
- 리스트의 메서드를 구현할 수 있는가
- 예외상황에 대한 처리를 할 수 있는가

과제-4

-2차원 리스트 이동경로 구현하기

10×10 미로가 주어진다.

0은 이동 가능, 1은 벽, 2는 먹이이다.

개미는 (2, 2)에서 출발하며, 이동한 위치는 9로 표시한다.

이동 규칙:

1. 오른쪽으로 갈 수 있으면 오른쪽으로 이동
2. 오른쪽이 막혀 있으면 아래쪽으로 이동
3. 먹이를 찾거나 더 이상 이동할 수 없으면 종료

입력 예시

```
1 1 1 1 1 1 1 1 1 1
1 0 0 1 0 0 0 0 0 1
1 0 0 1 1 1 0 0 0 1
1 0 0 0 0 0 0 1 0 1
1 0 0 0 0 0 0 1 0 1
1 0 0 0 0 1 0 1 0 1
1 0 0 0 0 1 2 1 0 1
1 0 0 0 0 1 0 0 0 1
1 0 0 0 0 0 0 0 0 1
1 1 1 1 1 1 1 1 1 1
```

출력 예시

```
1 1 1 1 1 1 1 1 1 1
1 9 9 1 0 0 0 0 0 1
1 0 9 1 1 1 0 0 0 1
1 0 9 9 9 9 9 1 0 1
1 0 0 0 0 0 9 1 0 1
1 0 0 0 0 1 9 1 0 1
1 0 0 0 0 1 9 1 0 1
1 0 0 0 0 1 0 0 0 1
1 0 0 0 0 0 0 0 0 1
1 1 1 1 1 1 1 1 1 1
```

고급 과제-1

-문자열 압축, 복원 프로그램

문자열을 연속된 문자 단위로 압축하고, 압축된 문자열을 다시 원래 문자열로 복원하는 프로그램을 작성하시오.

예시) 압축 : aaabbccccc → a3b2c4d1, 복원 : a3b2c4d1 → aaabbccccc

구현 함수

1. compress(text)
2. decompress(code)
3. is_valid_code(code)

is_valid_code

- 압축 문자열이 올바른 형식인지 검사한다.
- decompress 실행 전 검증 역할로 구현하시오 (잘못된 압축문자열 입력 시 ERROR 반환)

조건

1. 반복 횟수는 반드시 1 이상의 정수이다.
2. 반복 횟수는 두 자리 이상일 수 있다. 예: a12
3. 압축 대상 문자는 알파벳이라고 가정한다.
4. 숫자는 반복 횟수로만 사용된다.
5. decompress()는 잘못된 입력을 받으면 ERROR를 반환해야 한다.
6. 압축 결과가 원본보다 길어지면 원본을 반환한다.

입력 예시

```
print(compress("aaabbccccc"))
print(decompress("a3b2c4d1"))
print(decompress("a12b3"))
print(decompress("a0"))
print(decompress("3a"))
Print(compress("abcde"))
```

출력 예시

```
a3b2c4d
aaabbccccc
aaaaaaaaaaaabbb
ERROR
ERROR
abcde
```


고급 과제-2

-LLM 코드 리뷰 및 개선 보고서 작성하기

다음 과제 문제를 LLM에 입력하고 첫 번째 출력 코드를 수정하지 않은 채로 레퍼런스 코드로 저장하시오.

이후 직접 코드를 실행해보고 오류가 발생한다면 직접 오류 분석 및 오류 해결, 개선점이 있을 시 개선 가능성을 분석한 뒤 요구사항을 모두 만족할 수 있도록 코드를 개선하시오.

제출 목록

1. LLM 입력 프롬프트
2. 프롬프트 입력 후 LLM 첫 출력 코드 (.py 형태)
3. 오류 및 개선점 분석
4. 개선한 코드 (.py)
5. LLM 프롬프트 입력 단계부터 오류 개선까지의 고찰

보고서 형식은 pdf로 작성하고 레퍼런스 코드, 개선 코드를 함께 압축하여 제출

고급 과제-2

LLM 입력 문제 : 명령어 기반 도서 대출 관리 시스템

add 책이름 수량

- 책 등록, 이미 있으면 수량 추가

borrow 사용자 책이름

- 책이 있고 수량이 남아 있으며, 같은 사용자가 같은 책을 대출 중이 아닐 때만 대출
- 실패 시 ERROR + 사유 출력

return 사용자 책이름

- 실제로 대출 중인 책만 반납 가능
- 실패 시 ERROR + 사유 출력

status 책이름

- 남은 수량 출력
- 책이 존재하지 않으면 ERROR 출력

user 사용자

- 사용자의 대출 목록 출력
- 없으면 EMPTY

list

- 전체 책 목록을 등록 순서대로 출력

Exit

- 종료

조건:

- 책과 사용자 이름에는 공백 없음
- 수량은 0 이상의 정수
- 잘못된 명령어 형식은 ERROR
- eval() 사용 금지
- 함수로 기능 분리

과제 제출

과제 제출 형식:

- 과제 1~4, 고급과제 1은 (문제 번호.py)로 제출
- 고급과제 2는 보고서(pdf) 및 .py파일 2개 제출

제출 형태 자유

- 깃허브 등록 후 깃허브 주소
- 파일 압축 후 메일 제출 (inyupbaek@kw.ac.kr)

모든 코드에는 최소한의 주석 설명 작성

LLM 사용 가능. 하지만 고급과제 2와 같이 본인만의 고찰 및 코드 이해 필수 (고급과제 2 제외 보고서는 작성하지 않아도 됨)

기한

화요일 출근 전

Thank you
